Personal Information

Name: **Andrei Dimandar**

Student ID: **1002837**

Email: **andrei.dimandar@student.uni.lu**

Submitted on: **23.04.2025**

GitHub: https://github.com/AndreiDimandar/Thesis

Data Context

The data used in this research consists of three datasets collected from Dutch primary schools. The first dataset captures student preferences, gathered through surveys in which students indicate peers they would like to be grouped with. The second dataset contains additional student information, such as gender and the need for extra educational support. The third dataset includes constraints on teacher preferences, specifying conditions such as mandatory or restricted student pairings and ensuring balanced gender distribution.

These datasets originate from two earlier research initiatives conducted in collaboration with schools. Teachers collect student preferences through surveys, while additional student information is recorded based on school records. Constraints on teacher preferences are determined by the teacher's familiarity with their classrooms. The data is stored in an Excel file and prepared for classroom formation.

Data Description

In the preprocessing, we analyzed 90 students of the student grouping dataset, we found that the dataset consists of 17 students, with 43 boys and 34 girls. The students are nearly equally distributed between grade 4 (45 students) and grade 5 (45 students). Most students provided their preferences, with one student giving none and five students providing between one and four preferences. Around 20% of the student preferences reciprocate another student's preference, suggesting a moderate level of mutual preference.

The teacher preferences dataset covers 17 students, aligning with the student data, and includes 5 teacher-specified pairings. Notably, no pairings involve students in grade 5, highlighting a key issue, as it is the only grade where not all students were sorted with at least one of their preferences. The scores for absolute difference and relative scores showed discrepancies between the actual and class group distributions.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt
import networkx as nx
%matplotlib inline
# Load the student data from the Excel file
student_df = pd.read_excel('./data/Preferences.xlsx', sheet='1')
teacher_df = pd.read_excel('./data/Preferences.xlsx', sheet='2')
# Display the first few rows of each dataset to verify loading
student_df.head(), teacher_df.head()
```

```
(   Student  Group  Preference 1  Preference 2  Preference 3  Preference 4
0  S001     4      S002          S003          S004          NaN
1  S002     4      S001          S005          S006          NaN
2  S003     4      S009          S010          S004          NaN
3  S004     4      S003          S001          S005          NaN
4  S005     4      S006          S001          S002          NaN,
   Teacher  Preference 1  Preference 2  Preference 3  Preference 4  Preference 5
0  S015     S016          NaN           NaN           NaN           NaN
1  S016     S015          NaN           NaN           NaN           NaN
2  S017     S018          NaN           NaN           NaN           NaN
3  S018     S017          NaN           NaN           NaN           NaN
4  S019     S020          S021          NaN           NaN           NaN)
```

```
# Check for any missing values in the student data
print("Student data missing values:")
print(student_df.isnull().sum())
# Check for any missing values in the teacher data
print("\nTeacher data missing values:")
print(teacher_df.isnull().sum())
```

```
Student data missing values:
Student        0
Group          0
Preference 1   0
Preference 2   1
Preference 3   2
Preference 4   4
dtype: int64

Teacher data missing values:
Teacher        0
Preference 1   2
Preference 2   3
Preference 3   3
Preference 4   3
Preference 5   3
dtype: int64
```

```
# Check for duplicate entries
print("Duplicates in student dataset:")
print(student_df.duplicated().sum())
print("\nDuplicates in teacher dataset:")
print(teacher_df.duplicated().sum())
```

```
Duplicates in student dataset:
0

Duplicates in teacher dataset:
0
```

```
# Basic information about the student dataset
print("Student DataFrame info:")
student_df.info()
```

```
Student DataFrame info:
<class pandas.core.frame.DataFrame>
RangeIndex: 90 entries, 0 to 89
Data columns (total 6 columns):
 #   Student       90 non-null     object
 #   Group         90 non-null     int64
 #   Preference 1  90 non-null     object
 #   Preference 2  89 non-null     object
 #   Preference 3  88 non-null     object
 #   Preference 4  86 non-null     object
dtypes: int64: 1, object: 5
memory usage: 4.3 KB
```

```
# Basic information about the teacher dataset
print("Teacher DataFrame info:")
teacher_df.info()
```

```
Teacher DataFrame info:
<class pandas.core.frame.DataFrame>
RangeIndex: 17 entries, 0 to 16
Data columns (total 6 columns):
 #   Teacher       17 non-null     object
 #   Preference 1  15 non-null     object
 #   Preference 2  14 non-null     object
 #   Preference 3  14 non-null     object
 #   Preference 4  14 non-null     object
 #   Preference 5  14 non-null     object
dtypes: object: 6
memory usage: 1.0 KB
```

1. Corpus

```
# Count of the datasets
print("Dataset counts:")
print("Students:", len(student_df))
print("Teachers:", len(teacher_df))

# Number of students in each dataset
print("\nStudents per group:")
print(student_df['Group'].value_counts())

print("Constraints:", constraints.shape[0])

# Check for missing values in preferences
print("Missing preferences in each dataset:")
print("Students:", student_df.shape[0] - student_df['Preference 1'].count())
print("Teachers:", teacher_df.shape[0] - teacher_df['Preference 1'].count())

# Check for duplicate entries
print("Duplicates in student dataset:")
print(student_df.duplicated().sum())
print("Duplicates in teacher dataset:")
print(teacher_df.duplicated().sum())

# Check for self-preferences
print("Self-preferences in student dataset:")
self_pref = student_df[student_df['Preference 1'] == student_df['Student']]
print(self_pref[['Student', 'Preference 1']])

# Check for valid teacher preferences
print("Invalid teacher preferences:")
invalid_teacher = teacher_df[~teacher_df['Teacher'].isin(student_df['Student'])]
print(invalid_teacher[['Teacher', 'Preference 1']])
```

```
Dataset counts:
Students: 90
Teachers: 17

Students per group:
Group
4    45
5    45
Name: count, dtype: int64
Constraints: 5
Missing preferences in each dataset:
Students: 0
Teachers: 2

Duplicates in student dataset:
0
Duplicates in teacher dataset:
0
Self-preferences in student dataset:
Invalid teacher preferences:
Teacher, Preference 1]
Teacher  Preference 1
Teacher, Preference 1]
Teacher  Preference 1
Teacher, Preference 1]
Teacher  Preference 1
```